

Claude

Wednesday, September 30, 2026 · CLAUDE

Let us follow the guidelines and finish the assigned works.
Can you tell me what are the errors in this project and in what order we will resolve them. We will commit changes one by one to my repo like they tell us to do.

I read through both files. There is exactly **one real bug**. The other three items are intentionally empty stubs, and they are the features to add.

What's wrong

#	Item	Type	Location
1	Mushrooms die after 3 hits instead of 4	Bug	<code>hit_mushroom</code>
2	<code>mushroom_color(hp)</code> returns nothing (<code>pass</code>)	Feature stub	top of <code>game.py</code>
3	<code>on_segment_hit(segment, score)</code> returns nothing	Feature stub	top of <code>game.py</code>
4	<code>wave_speed_bonus(wave)</code> returns nothing	Feature stub	top of <code>game.py</code>

The bug: `hit_mushroom` decrements HP and then checks `<= 1`. HP goes $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$, so the mushroom is deleted on the **third** hit. The check should be `<= 0`. This also means the darkest colour in the default fade (HP 1) is never drawn.

Until Task 4 is done, the README's "next, faster wave" isn't true. Later waves only get one extra segment, and the speed stays the same.

Order and commits

1. **Record the "before" video first.** Shoot one mushroom three times and show it vanishing. This is a required submission item, and it's impossible to capture after the fix.
2. **Fix** `hit_mushroom`. Commit: `Fix mushroom durability off-by-one (<= 1 -> <= 0)`. This comes first because Task 2 depends on HP values 1 through 4 actually occurring.
3. **Implement** `mushroom_color(hp)`. Commit: `Add mushroom hit-flash colour`.
4. **Implement** `on_segment_hit`. Commit: `Add segment-hit effect`.
5. **Implement** `wave_speed_bonus`. Commit: `Scale centipede speed with wave`. This is the simplest one, e.g. `1 + 0.15 * (wave - 1)`.
6. **Record the "after" video** showing the 4-hit mushroom, the colour effect, the hit effect, and a faster second wave. Then save your chat link.

Two design catches to discuss with me

- **Task 2:** `mushroom_color` only receives `hp`, so it can't know whether the mushroom was *just* hit. A "flash white on hit" needs a little extra state, such as a set of recently hit cells that `hit_mushroom` fills and `draw` clears. An alternative is to make the colour depend on HP alone.
- **Task 3:** the return value is ignored and `score` is passed by value. A spark effect or a bonus therefore can't be applied from inside the function alone. We'd need a small effects list on `Game` that gets drawn in `draw`, or we can keep the hook simple, such as a sound or a print.

Want to start with Task 1? It's a one-line change, and I can give you the exact diff and commit command.



ok lets start with task 1

Task 1: Fix the mushroom durability bug

In `game.py`, find `hit_mushroom` and change one character:

python

```
def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 0:    # was: <= 1
        del self.mushrooms[cell]
    self.score += 5
```

Why it works: a mushroom starts at `MUSHROOM_HP = 4`. Each hit subtracts 1, so the values go 3, 2, 1, 0. With `<= 0` it's deleted on the fourth hit, and HP 1 is now a state the mushroom actually spends time in.

Before you change it

Record your 10-second "before" video first. Run `python game.py`, shoot one mushroom, and count: it disappears on the third hit. Once you edit the file, you can't easily get that behavior back for the video (unless you use `git stash` or check out the old commit).

Test after the fix

1. Run the game and shoot a single mushroom.
2. Hits 1 and 2 should visibly darken it. The third hit should now leave it on screen in its darkest shade, and only the fourth hit should clear it.
3. Score should go up by 5 only on that fourth hit.

Note on centipede-created mushrooms: when you shoot a body segment, `split_chain` places a fresh mushroom at `MUSHROOM_HP` (4), so those also take four hits now. That's the intended behavior.

Commit

bash

```
git add game.py
git commit -m "Fix mushroom durability off-by-one (<= 1 -> <= 0)"
git push
```

Tell me what you see when you test it, and then we'll move on to Task 2. That one needs a design decision, since `mushroom_color` only receives `hp`, so we'll need to pick how to handle the "flash on hit" idea.

📄 ↻ 👍 🗨️ ↺ ...

ok give me task 2

Task 2: `mushroom_color(hp)`

Design decision

As I mentioned earlier, `mushroom_color` only receives `hp`, so it can't know which mushroom was *just* hit. There are two options:

- **HP-only colors (recommended):** each HP level gets its own color, so damage is instantly readable. It fits the README's call signature exactly, and no other code needs to change.
- **True "flash white on hit":** this needs extra state on `Game` (a timer per cell) plus a change in `draw`, and at 60 FPS a one-frame flash is barely visible without a timer. It's a fine extension, but it goes beyond what the function can do on its own.

I suggest committing the HP-only version now. I can show the flash as a bonus afterward if you want it.

The change

Replace the stub at the top of `game.py`:

python

```
MUSHROOM_COLORS = {
    4: (200, 80, 170),    # healthy: magenta
    3: (230, 140, 80),    # orange
    2: (230, 90, 60),     # red-orange
    1: (255, 50, 50),     # about to break: bright red
}

def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit
    points, or None for the default."""
    return MUSHROOM_COLORS.get(hp)
```

Why this works

- `draw` calls `mushroom_color(hp) or (default...)`. Returning `None` for an unexpected HP (say you change `MUSHROOM_HP` later) safely falls back to the default fade.
- HP 1 is now a state you can see, and it's the red "one more hit" warning. That depends on the Task 1 fix, which is why we did it first.
- The dictionary lookup keeps the function tiny and easy to explain in your write-up.

Test

1. Run the game and shoot a single mushroom four times.
2. You should see magenta, then orange, then red-orange, then bright red, and it should vanish on the fourth hit.

Commit

bash

```
git add game.py
git commit -m "Add per-HP mushroom colours"
git push
```

Let me know what you see. Then we'll move on to Task 3, which has the "effects list" design question. Or say the word if you'd like the timed white-flash extension first.

     ...

how does it work ecaxtly

How Task 2 works, step by step

1. `draw` runs every frame

For every mushroom on the field, `draw` runs this line:

python

```
color = mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
```

`self.mushrooms` is a dictionary like `{(row, col): hp}`, so each mushroom's remaining HP is passed into `mushroom_color`.

2. The dictionary is a lookup table

python

```
MUSHROOM_COLORS = {4: (200, 80, 170), 3: (230, 140, 80), 2: (230, 90, 60), 1: (255, 50, 50)}
```

The keys are HP values and the values are `(r, g, b)` colors. Each color is three numbers from 0 to 255 for red, green, and blue.

3. `.get(hp)` does the lookup

python

```
return MUSHROOM_COLORS.get(hp)
```

- If `hp` is a key in the table (1 to 4), it returns that color tuple.
- If it isn't, `.get` returns `None` instead of crashing. Plain `MUSHROOM_COLORS[hp]` would raise a `KeyError`.

4. `or` provides the fallback

In Python, `a or b` returns `a` if `a` is truthy, otherwise `b`.

- A color tuple like `(230, 140, 80)` is truthy, so `draw` uses it.
- `None` is falsy, so `draw` falls back to the original formula after `or`.

That's why returning `None` means "use the default" in the README.

Example

You shoot a mushroom once. In `hit_mushroom`, its HP goes from 4 to 3. On the next frame, `draw` reads `hp = 3`, `mushroom_color(3)` returns `(230, 140, 80)`, and `pygame.draw.circle` paints the cap orange. Nothing else in the game needs to know about colors. The HP value alone decides what you see.

Why it depends on Task 1

Before the fix, the mushroom was deleted when HP reached 1, so the `1: (255, 50, 50)` entry would never have appeared on screen. With `<= 0`, that red "one hit left" state now shows.

Is there a specific part you'd like me to go deeper on, like the `or` trick or how tuples become colors?

ok give me task 3

Task 3: `on_segment_hit(segment, score)`

Design decision

The hook is a module-level function. It can't touch `Game`, and its return value is ignored. So the function only *creates* the effect (sparks) and puts it in a shared list. `Game` then updates and draws that list. That's four small edits, all in `game.py`.

1. Add `import math` at the top

python

```
import math
import random
import pygame
```

2. Replace the stub

python

```
sparks = [] # each spark: [x, y, vx, vy, seconds_left]

def on_segment_hit(segment, score):
    """Spawn a burst of sparks where a centipede segment was shot."""
    cx = segment.col * CELL + CELL // 2
    cy = segment.row * CELL + CELL // 2
    for _ in range(12):
        angle = random.uniform(0, 2 * math.pi)
        speed = random.uniform(60, 180)
        sparks.append([cx, cy, math.cos(angle) * speed,
math.sin(angle) * speed, 0.4])
```

3. Update sparks in `Game.update`

Add this right after the `if self.state != "play": return` check:

python

```
for s in sparks:
    s[0] += s[2] * dt
    s[1] += s[3] * dt
    s[4] -= dt
sparks[:] = [s for s in sparks if s[4] > 0]
```

4. Draw sparks in `Game.draw`

Add this after the loop that draws the chains and before the bullet is drawn:

python

```
for x, y, _, _, life in sparks:
    pygame.draw.circle(screen, (255, 220, 120), (int(x), int(y)), 2)
```

5. Clear sparks on reset

Add this as the first line of `Game.reset`:

python

```
sparks.clear()
```

How it works

- When a segment is shot, `split_chain` calls `on_segment_hit`. The function turns the segment's grid cell into pixel coordinates and adds 12 sparks. Each has a random direction, a random speed, and 0.4 seconds to live.
- Every frame, `update` moves each spark by `velocity * dt` and reduces its lifetime. Expired sparks are removed. Writing `sparks[:] = ...` changes the list in place, so the module-level list stays the same object.
- `draw` paints each remaining spark as a small yellow dot.

Limitation

The hook receives the segment and the score, but not whether it was a head shot, so every hit gets the same effect. That's fine for this task. The `score` parameter goes unused, which is allowed by the README.

Test

1. Run the game and shoot a body segment. A burst of sparks should fly out from that cell and fade in under half a second.
2. Press `R` mid-burst. The sparks should disappear.

Commit

bash

```
git add game.py
git commit -m "Add spark effect on segment hit"
git push
```

Tell me what you see, and then we'll do Task 4, the speed scaling.

📄 🔗 👍 🗨️ ↻ ...

ok give me task 4

Task 4: `wave_speed_bonus(wave)`

This one is a single function. Replace the stub at the top of `game.py`:

python

```
def wave_speed_bonus(wave):  
    """Return an extra tick-rate multiplier for centipede segments at  
    the given wave, or None for the default speed."""  
    return 1 + 0.15 * (wave - 1)
```

How it works

`update` uses the return value like this:

python

```
effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
```

`TICK` is the time between segment steps (0.09 seconds). Dividing it by a multiplier greater than 1 makes the tick shorter, so segments step more often.

Wave	Multiplier	Effective tick	Steps per second
1	1.00	0.090 s	~11.1
2	1.15	0.078 s	~12.8
3	1.30	0.069 s	~14.4
5	1.60	0.056 s	~17.8
10	2.35	0.038 s	~26.1

Wave 1 returns exactly `1.0`, so the starting speed is unchanged. Each later wave adds 15% of the base speed.

About the `or 1` fallback

The function never returns `None` or `0` now, so the `or 1` in `update` is never used. It only matters if you later return `None` to mean "default speed."

One thing to watch at high waves

The `while self.timer >= effective_tick` loop can run several steps in one frame if the tick gets very small. `dt` is capped at 0.05 s, so at wave 10 (tick 0.038 s) you'd occasionally get two steps in one frame. That's fine for this game. If you wanted to cap the difficulty, you could use `min(2.5, 1 + 0.15 * (wave - 1))`.

Test

1. Run the game and clear the first wave. It's the hardest part to test, since you need to shoot every segment.
2. To test faster, temporarily change `1 + 0.15 * (wave - 1)` to `1 + 2 * (wave - 1)`, or start with `self.wave = 5` in `reset`. Revert it before

committing.

3. Wave 2 should visibly move quicker than wave 1, and the HUD should show the wave number.

Commit

bash

```
git add game.py
git commit -m "Scale centipede speed with wave"
git push
```

Final step: the "after" video

Record 10 seconds showing all four changes:

1. A mushroom changing color through four hits and vanishing on the fourth.
2. Sparks when you shoot a segment.
3. Ideally, a faster wave. If clearing wave 1 takes too long, use the temporary `self.wave = 5` trick and mention it in your write-up.

Then save your chat link with the full history, and your submission is complete.

